

Learning Disorder Detection System

An early-stage, multimodal deep learning system for learning-disorder screening and end-user assistance, with dyslexia as the primary use case and emphasis on Bengali, multilingual, and low-resource settings.

For a fuller project walkthrough, see docs/PROJECTDOCUMENTATION.md (/d:/Project/DyslexiaDetectionSystem/docs/PROJECTDOCUMENTATION.md).

For a full dataset guide, see docs/DATASETS.md (/d:/Project/DyslexiaDetectionSystem/docs/DATASETS.md).

For a paper-style draft based on the current implementation, see docs/RESEARCHPAPERDRAFT.md (/d:/Project/DyslexiaDetectionSystem/docs/RESEARCHPAPERDRAFT.md).

For a 28-slide presentation deck for demonstrations, see docs/DyslexiaDetectionSystemDemoDeck.pptx (/d:/Project/DyslexiaDetectionSystem/docs/DyslexiaDetectionSystemDemoDeck.pptx).

The codebase supports handwriting images, reading audio features, multilingual text samples, reading-behavior indicators, explainability hooks, and lightweight deployment through a local Streamlit dashboard.

For full web deployment, see docs/DEPLOYMENT.md (/d:/Project/DyslexiaDetectionSystem/docs/DEPLOYMENT.md).

Deploy link: <https://learning-disorder-intelligence.onrender.com/> (<https://learning-disorder-intelligence.onrender.com/>)

For ready-to-upload biomarker test CSVs, see docs/BIOMARKERTESTDATA.md (/d:/Project/DyslexiaDetectionSystem/docs/BIOMARKERTESTDATA.md).

Live demo: <https://learning-disorder-intelligence.onrender.com/> (<https://learning-disorder-intelligence.onrender.com/>)

Release notes: RELEASENOTES.md (/d:/Project/DyslexiaDetectionSystem/RELEASENOTES.md)

Web Deployment Checklist

1. Push the latest main branch to GitHub.
1. Choose one host: Render, Railway, or Fly.io.
 1. Use the matching config file from render.yaml (/d:/Project/DyslexiaDetectionSystem/render.yaml), railway.json (/d:/Project/DyslexiaDetectionSystem/railway.json), or fly.toml (/d:/Project/DyslexiaDetectionSystem/fly.toml).
 1. Ensure the service exposes port 10000 and serves /healthz.
 1. Open the public HTTPS URL and verify the microphone flow in the browser.
 1. If transcription is slow on the first request, wait for Whisper to finish loading the model.

Render Auto-Deploy Notes

The Render setup in this repository is configured for automatic deploys:

- render.yaml sets autoDeploy: true
- the container listens on port 10000
- the health check path is /healthz

That means a push to the connected GitHub branch, usually main, should trigger a fresh Render build automatically if the Render service is already linked to this repo.

If the site does not refresh after a push, check these Render settings:

1. The service is connected to archisman-das/Learning-Disorder-Intelligence
1. The deploy branch is main
1. Auto-deploy is enabled in the Render dashboard
1. The latest build did not fail during image build or health check

At A Glance

Area	Summary
Core goal	Multimodal learning-disorder screening and educational support
Main languages	Bengali, English, multilingual
Primary inputs	Handwriting, audio, text, behavior, eye-tracking, biomarker CSVs
Main outputs	Screening labels, therapy scores, reports, local records, biomarker rankings
Main UIs	Streamlit dashboard, local web dashboard, React frontend
Deployment style	Local-first, offline-friendly, research-friendly

Dashboard Surfaces

Surface	Entry point
Streamlit dashboard	[`app.py`](/d:/Project/Dyslexia_Detection_System/app.py)
Standalone web dashboard	[`run_local_web.py`](/d:/Project/Dyslexia_Detection_System/run_local_web.py) + [`web/`](/d:/Project/Dyslexia_Detection_System/web/)
React frontend	[`frontend/src/App.jsx`](/d:/Project/Dyslexia_Detection_System/frontend/src/App.jsx)

Model Families

The current comparison snapshot uses three active supervised screening models:

- AttentionMultimodalModel via multimodal_attention
- TransformerMultimodalModel
- ViTMultimodalModel

Legacy baselines such as InitialCNNModel, InitialLSTMMModel, and InitialCNNLSTMMModel remain in the codebase for experimentation and historical comparison, but they are no longer the primary ranked set in the docs or dashboard.

Model family	Purpose	Strength	Limitation
`MultimodalDyslexiaModel`	Default screening model	Balanced and practical	Uses concatenation fusion
`TransformerMultimodalModel`	Transformer text branch	Better context modeling	Heavier than GRU version
`ViTMultimodalModel`	Patch-based handwriting branch	Better image structure modeling	More compute
`AttentionMultimodalModel`	Learned modality weights	Better interpretability	More tuning-sensitive
`BengaliLearningDisorderFoundationModel`	Shared foundation encoder	Reusable across tasks	Needs more data and pretra

Model Comparison Matrix

The current selection pipeline keeps the live test flow separate from the model-statistics hub. The table below is a documentation snapshot of the latest cross-validation selection run on the tough benchmark manifest, so the metrics are easy to read in one place.

Note: the standalone web dashboard uses display-only label swapping in some model-statistics panels, so vit and transformer can appear exchanged in the UI while the underlying model identifiers in this documentation stay unchanged.

Selection value is computed as a weighted performance score:

$$\text{selection_value} = (0.5 \text{ CV F1}) + (0.3 \text{ CV Accuracy}) + (0.2 * \text{CV Precision}) + \text{model priority bonus}$$

Model	CV Accuracy	CV Precision	CV Recall	CV F1	CV Balanced Acc	Selection Value	Rank
`multimodal_attention`	95.0%	93.3%	100.0%	96.0%	95.0%	1.202	1
`transformer`	91.7%	87.8%	93.3%	89.8%	91.7%	1.080	2
`vit`	80.0%	76.7%	86.7%	80.2%	80.0%	0.954	3

Validation Matrix

The validation matrix below summarizes the same model families from the cross-validation view without the holdout column. This is useful when you want to compare the model behavior before final test-set selection.

Model	CV Accuracy	CV Precision	CV Recall	CV F1	CV Balanced Acc
`multimodal_attention`	95.0%	93.3%	100.0%	96.0%	95.0%
`transformer`	91.7%	87.8%	93.3%	89.8%	91.7%
`vit`	80.0%	76.7%	86.7%	80.2%	80.0%

Pipeline ranking used in the current selection snapshot:

- 1. multimodal_attention
- 1. transformer
- 1. vit

Screenshot Slots

This repository does not currently ship captured dashboard screenshots. If you want to add them later, place the image files here and reference them in this README:

Suggested file	Dashboard
[`docs/screenshots/streamlit-dashboard.png`](/d:/Project/Dyslexia_Detection_System/docs/screenshots/streamlit-dashboard.png)	Main Streamlit
[`docs/screenshots/web-dashboard.png`](/d:/Project/Dyslexia_Detection_System/docs/screenshots/web-dashboard.png)	Local browser
[`docs/screenshots/react-dashboard.png`](/d:/Project/Dyslexia_Detection_System/docs/screenshots/react-dashboard.png)	React/Vite frontend

For now, the architecture and feature tables below give a reliable text reference until real captures are added.

Features

- Bengali, English, and multilingual dataset manifest support
- Handwriting image preprocessing and augmentation
- Reading-audio log-spectrogram extraction
- Language-aware text normalization and character-level tokenization
- Reading-behavior features for timing, hesitation, repetition, and omission patterns
- Multimodal PyTorch model for dyslexia-risk screening
- Grad-CAM utility for handwriting explainability
- Multilingual speech-therapy exercise tracking and adaptive recommendations
- Reinforcement learning-based adaptive tutoring with persistent policy updates
- Personalized intervention recommendation system for reading, pronunciation, and spelling exercises
- Attention-based multimodal fusion with per-prediction modality importance scores
- Digital biomarker discovery for handwriting, speech, and reading behavior signals
- Bengali eye-tracking dataset collection with fixation/regression/gaze biometrics
- Demo dataset generator for smoke testing
- Offline-friendly Streamlit visualization dashboard

Project Structure

```
.
|-- app.py
|-- requirements.txt
|-- scripts/
|   |-- create_demo_dataset.py
|   |-- start_streamlit.ps1
|-- src/
|   |-- dyslexia_detection/
|       |-- config.py
|       |-- dataset.py
|       |-- explainability.py
|       |-- models.py
|       |-- preprocessing.py
|       |-- schemas.py
|       |-- train.py
```

Dataset Manifest

Training data is described with a CSV manifest. Each row represents one anonymized user sample.

```
sample_id,student_hash,handwriting_path,audio_path,text_sample,spelling_errors,pronunciation_errors,reading_time_seconds,hesitation_count,omission_count
S001,anon_001,handwriting/S001.png,audio/S001.wav,ami bangla poro,2,1,30.0,3,2,1,1
```

Labels:

- 0: low risk
- 1: moderate/high risk

Required columns:

- sample_id
- student_hash
- handwriting_path
- audio_path
- text_sample
- spelling_errors
- pronunciation_errors
- label

Recommended behavior columns:

- readingtimesseconds
- hesitation_count
- repetition_count
- omission_count

Only anonymized identifiers should be stored. Do not store names, phone numbers, school IDs, or addresses in the manifest.

Dataset Creation Workflow

Create the collection workspace:

```
python scripts/setup_dataset_workspace.py --root data/collection
```

This creates folders for raw handwriting, raw audio, augmented files, split manifests, and a collection protocol document.

Gather one handwriting/text/audio sample from the command line:

```
python scripts/add_sample_to_manifest.py --workspace data/collection --sample-id S001 --student-hash anon_001 --handwriting-file p
```

You can also use the dashboard's Sample Collection tab to upload handwriting and audio files, choose the sample language, enter text and reading observations, and append the row directly into `data/collection/manifest.csv`.

Create a blank manifest template:

```
python scripts/create_collection_template.py --output data/collection/manifest_template.csv
```

Validate collected data:

```
python scripts/validate_manifest.py --manifest data/collection/manifest.csv
```

Clean, normalize, augment, and split the collected dataset:

```
python scripts/prepare_collected_dataset.py --manifest data/collection/manifest.csv --output-root data/collection/processed --split
```

This pipeline:

- cleans duplicate or malformed manifest rows
- normalizes text whitespace with language-aware Unicode handling
- standardizes numeric error and behavior fields
- normalizes handwriting images to fixed grayscale canvas size
- normalizes reading audio to 16 kHz WAV with trimmed silence and consistent volume
- creates handwriting and audio augmentations
- writes a final `prepared_manifest.csv`
- optionally writes train/validation/test splits

Anonymize user identifiers before training:

```
python scripts/anonymize_manifest.py --input data/collection/manifest.csv --output data/collection/manifest_anonymized.csv --salt YC
```

Create handwriting augmentation:

```
python scripts/augment_handwriting_dataset.py --manifest data/collection/manifest_anonymized.csv --output-manifest data/collection
```

Create reading-audio augmentation:

```
python scripts/augment_audio_dataset.py --manifest data/collection/augmented_manifest.csv --output-manifest data/collection/audio
```

Create train/validation/test splits:

```
python scripts/split_manifest.py --manifest data/collection/audio_augmented_manifest.csv --output-dir data/collection/splits
```

The dashboard's Dataset Creation tab shows the required schema, current validation status, and basic quality metrics.

Quick Start

Create a demo dataset:

```
python scripts/create_demo_dataset.py
```

Train a small model:

```
python -m src.dyslexia_detection.train --manifest data/demo/manifest.csv --epochs 2
```

Discover digital biomarkers associated with dyslexia:

```
python scripts/discover_biomarkers.py --manifest data/demo/audio_augmented_manifest.csv --output-dir reports/biomarkers --top-k
```

Collect Bengali eye-tracking dataset from webcam:

```
python scripts/collect_eye_tracking_webcam.py --sample-id ET001 --participant-hash anon_user_001 --language Bengali --prompt "আমি একটি বাংলা বাক্য পড়ছি।"
```

This writes:

- data/collection/eyetracking/ET001gaze_trace.csv
- data/collection/eyetracking/eyetracking_metrics.csv

with fixation duration, regressions, reading speed, and gaze-pattern features.

Train the initial CNN/LSTM baseline:

```
python scripts/train_initial_cnn_lstm.py --manifest data/demo/audio_augmented_manifest.csv --epochs 5 --batch-size 8
```

Train all initial baselines, CNN-only, LSTM-only, and CNN/LSTM:

```
python scripts/train_initial_models.py --manifest data/demo/audio_augmented_manifest.csv --epochs 5 --batch-size 8
```

Train Transformer, ViT, and ViT+Transformer multimodal models:

```
python scripts/train_advanced_models.py --manifest data/demo/audio_augmented_manifest.csv --epochs 3 --batch-size 8
```

Train every supported model family in one pass:

```
python scripts/train_all_models.py --manifest data/demo/audio_augmented_manifest.csv --epochs 5 --batch-size 8 --text-language mu
```

For best results, point `--manifest` at your largest cleaned and anonymized labeled dataset. More real samples usually helps accuracy and makes confidence values more reliable.

Run family-safe cross-validation to get a more trustworthy accuracy/F1 estimate:

```
python scripts/train_cross_validation.py --manifest data/demo/audio_augmented_manifest.csv --model multimodal_attention --task bin
```

This keeps augmented siblings in the same fold, repeats the k-fold split several times, trains each fold independently, and writes aggregated metrics to checkpoints/crossvalidation/crossvalidationresults.csv plus crossvalidation_summary.json.

Run a hard holdout test evaluation after model selection:

```
python scripts/train_holdout_evaluation.py --manifest data/demo/audio_augmented_manifest.csv --model multimodal_attention --task bin
```

This creates a train/validation/test split with a fully untouched test set, trains on train/validation only, and reports final holdout metrics from the test split.

Automatically pick the best model with repeated cross-validation, then run the hard holdout test:

```
python scripts/select_model_via_cv_and_holdout.py --manifest data/demo/audio_augmented_manifest.csv --task binary --text-language
```

This ranks model families by repeated cross-validation, selects the top performer, and then evaluates that winner once on an untouched test split.

For the strict family-safe benchmark, the default hard-split runner now targets the tougher balanced split:

```
python scripts/select_model_on_hard_split.py --train-manifest data/benchmarks/hard_family_split_balanced_harder/train.csv --validation
```

That split is the preferred sanity-check when you want a more realistic final F1 and confidence report.

You can also run the same strict benchmark in one step:

```
python scripts/run_strict_benchmark.py
```

Train attention-based multimodal fusion:

```
python -m src.dyslexia_detection.train --manifest data/demo/audio_augmented_manifest.csv --model multimodal_attention --epochs 5
```

Pretrain Bengali multimodal learning-disorder foundation model:

```
python scripts/train_foundation_model.py --manifest data/demo/audio_augmented_manifest.csv --epochs 5 --batch-size 8 --text-language
```

Fine-tune disorder-specific adapter (dyslexia / dysgraphia / dyscalculia):

```
python scripts/fine_tune_foundation_adapter.py --manifest data/demo/audio_augmented_manifest.csv --foundation-checkpoint checkpoint
```

Train with a multilingual text vocabulary for mixed-language manifests:

```
python -m src.dyslexia_detection.train --manifest data/collection/processed/prepared_manifest.csv --model vit_transformer --text-language
```

Train severity-level models (Mild, Moderate, Severe):

```
python scripts/train_severity_models.py --manifest data/demo/audio_augmented_manifest.csv --model vit_transformer --task severity -
```

Cross-lingual transfer learning (English -> Bengali):

```
python scripts/train_cross_lingual_transfer.py --manifest data/collection/processed/prepared_manifest.csv --english-checkpoint checkpoint
```

This transfers shared handwriting/audio/behavior/classifier knowledge from an English checkpoint, then fine-tunes on Bengali data with optional feature-level distillation for low-resource settings.

Train continuous severity regression:

```
python scripts/train_severity_models.py --manifest data/demo/audio_augmented_manifest.csv --model vit_transformer --task regression
```

Self-supervised Bengali audio representation learning (contrastive):


```
python scripts/pretrain_bengali_ssl.py --manifest data/demo/audio_augmented_manifest.csv --objective contrastive --epochs 5 --output c
```

Self-supervised masked modeling:

```
python scripts/pretrain_bengali_ssl.py --manifest data/demo/audio_augmented_manifest.csv --objective masked --epochs 5 --output c
```

Teacher-distillation with wav2vec2 or HuBERT:

```
python scripts/pretrain_bengali_ssl.py --manifest data/demo/audio_augmented_manifest.csv --objective wav2vec2 --teacher-model facebook/wav2vec2-large-batch19
python scripts/pretrain_bengali_ssl.py --manifest data/demo/audio_augmented_manifest.csv --objective hubert --teacher-model facebook/hubert-large-lstm-batch19
```

Fine-tune dyslexia model from SSL audio encoder:

```
python scripts/fine_tune_from_ssl.py --manifest data/demo/audio_augmented_manifest.csv --ssl-checkpoint checkpoints/ssl/audio_con
```

Or call the shared trainer directly:

```
python -m src.dyslexia_detection.train --manifest data/demo/audio_augmented_manifest.csv --model cnn_lstm --epochs 5 --batch-size 16
```

Run the dashboard:

```
python -m streamlit run app.py --server.port 8501 --server.address localhost
```

The dashboard includes dataset overview charts, sample inspection, model metrics, prediction analytics, reading-behavior visualization, and live screening.

Run the practical mobile/web prototype:

```
python -m streamlit run prototype_app.py --server.port 8502 --server.address localhost
```

The prototype includes quick screening, webcam capture analysis, sample collection, guided practice, speech-therapy support, and local record review. It uses exports/deployment/pruned30quantized.pt by default and falls back to checkpoints/best_model.pt.

Run federated learning across distributed client manifests:

```
python scripts/train_federated.py --client-manifests data/demo/audio_aug_splits/train.csv,data/demo/audio_aug_splits/validation.csv --
```

After each screening, the system now generates a personalized intervention plan and adapts future recommendations using a reinforcement-learning policy update.

Use the Webcam tab for browser/mobile camera capture. This is suitable for practical classroom demos because it works inside Streamlit without extra desktop camera windows.

Run continuous local webcam analysis with OpenCV:

```
python scripts/realtime_webcam_analysis.py --model exports/deployment/pruned_30_quantized.pt --camera 0 --interval 1.0 --text "am
```

Press q or Esc to stop the webcam window.

Log one offline speech-therapy session:

```
python scripts/record_speech_therapy_session.py --student-hash anon_student_001 --task-id bn_sentence_easy --audio-file path/to/se
```

Speech-therapy sessions are stored in `data/mobilecollection/therapy/therapysessions.csv` and include language-specific target prompts, audio paths, pronunciation observations, a therapy score, and a recommended next step.

Proposed System Architecture

The implementation follows the requested architecture as code:

- Input Layer: manifest rows, handwriting image paths, reading-audio paths, multilingual text, error counts, and reading-behavior values
- Preprocessing Layer: image normalization, audio log-spectrogram extraction, language-aware text tokenization, behavior tensor creation
- Feature Extraction Layer: CNN handwriting encoder, audio temporal encoder, behavior encoder, error features
- Sequence Modeling Layer: GRU/Transformer-based multilingual text sequence modeling
- Classification Layer: multimodal feature fusion and dyslexia-risk prediction
- Explainability Module: Grad-CAM handwriting attention overlay
- Deployment Layer: TorchScript and quantized TorchScript export

Run one sample through every architecture layer:

```
python scripts/run_architecture_pipeline.py --manifest data/demo/manifest.csv --sample-id S001
```

Objective Implementation Map

1. Develop a deep learning dyslexia screening model:

```
python -m src.dyslexia_detection.train --manifest data/demo/manifest.csv --epochs 2
```

1. Create multilingual-language compatible datasets:

```
python scripts/create_collection_template.py --output data/collection/manifest_template.csv  
python scripts/validate_manifest.py --manifest data/demo/manifest.csv
```

2a. Transfer English knowledge to Bengali:

```
python scripts/train_cross_lingual_transfer.py --manifest data/collection/processed/prepared_manifest.csv --english-checkpoint checkpoints/english/ssl/audio_conformer
```

1. Build lightweight and offline-compatible AI models:

```
python scripts/export_lightweight_model.py --output-dir exports  
python scripts/export_lightweight_model.py --output-dir exports --quantize
```

3a. Learn Bengali audio representations before fine-tuning:

```
python scripts/pretrain_bengali_ssl.py --manifest data/demo/audio_augmented_manifest.csv --objective contrastive --epochs 5 --output-dir checkpoints/ssl/audio_conformer  
python scripts/fine_tune_from_ssl.py --manifest data/demo/audio_augmented_manifest.csv --ssl-checkpoint checkpoints/ssl/audio_conformer
```

Apply pruning, quantization, TorchScript export, and benchmarking:

```
python scripts/optimize_for_deployment.py --checkpoint checkpoints/best_model.pt --output-dir exports/deployment --prune-amount
```

Run offline inference with the optimized artifact:

```
python scripts/run_lightweight_inference.py --model exports/deployment/pruned_30_quantized.pt --handwriting data/demo/handwriting
```

Predict severity class or score from a trained checkpoint:

```
python scripts/predict_severity.py --checkpoint checkpoints/severity_vit_transformer/best_model.pt --handwriting data/demo/handwriting
```

Inspect modality importance (handwriting/speech/text/reading behavior):

```
python scripts/predict_modality_attention.py --checkpoint checkpoints/multimodal_attention/best_model.pt --handwriting data/demo/handwriting
```

1. Implement explainable AI:

The dashboard's Explainability tab supports:

- CNN Grad-CAM handwriting overlays for CNN-based handwriting encoders
- ViT patch-attention handwriting overlays for ViT checkpoints
- Transformer token-attention charts for Bengali or multilingual text checkpoints

Useful checkpoint paths:

```
checkpoints/best_model.pt
checkpoints/transformer/best_model.pt
checkpoints/vit/best_model.pt
checkpoints/vit_transformer/best_model.pt
```

1. Design an assistive educational prototype:

The dashboard's Guided Practice tab provides Bengali, English, and mixed-language reading prompts, handwriting prompts, fluency tracking, speech-therapy prompts, and adaptive support messages.

1. Recommend personalized intervention plans:

The Live Screening and prototype Screen/Webcam tabs now recommend customized reading, pronunciation, and spelling exercises after detection, with weekly targets and adaptive policy updates logged under:

- data/collection/intervention/recommendations.csv (dashboard)
- data/mobile_collection/intervention/recommendations.csv (prototype)

Ethical Use

This project is intended for screening and assistive learning support, not clinical diagnosis. Final assessment should involve qualified professionals. Collect data only with informed consent from participants/guardians where applicable, and anonymize all personally identifiable information.

Web Dashboard (HTML/CSS/JS)

If Streamlit is not reachable, run the standalone web dashboard:

```
powershell -ExecutionPolicy Bypass -File scripts/start_web_dashboard.ps1
```

Then open:

<http://localhost:8080>

Professional Flask Dashboard

Run the backend-powered dashboard (project-feature integrated):

```
powershell -ExecutionPolicy Bypass -File "C:\Dyslexia_Detection_System\scripts\start_professional_dashboard.ps1"
```

Or double-click:

- C:\DyslexiaDetectionSystem\startprofessionaldashboard.bat

Open:

<http://127.0.0.1:5050>

React + Vite Frontend (Recommended UI)

This frontend uses React + Vite and proxies API calls to the Flask backend.

1. Start backend:

```
python dashboard_web.py
```

1. In another terminal:

```
cd frontend  
npm install  
npm run dev
```

1. Open:

<http://127.0.0.1:5173>

Backend connection notes:

- Dev mode: Vite proxies `/api/*` to <http://127.0.0.1:5050> (configured in frontend/vite.config.js).
- Optional direct API base: set `VITEAPIBASE` (example: <http://127.0.0.1:5050>) before `npm run dev`.
- Health check: <http://127.0.0.1:5050/api/health>
- Production mode: build frontend with `npm run build`; Flask serves frontend/dist automatically at <http://127.0.0.1:5050>.

Optional single command:

```
powershell -ExecutionPolicy Bypass -File "C:\Dyslexia_Detection_System\scripts\start_react_dashboard.ps1"
```

Direct Local Run (No PowerShell)

Run directly with Python:

```
python run_local_web.py
```

Or double-click:

- C:\DyslexiaDetectionSystem\startlocalweb.bat